

Processor Verification

Lab 7 - Formal Verification

Lorenzo Pattaro Zonta

June 4, 2026

1 Design Under Test: packet.sv

The packet module implements a finite state machine (FSM) that models the reception of a packet through successive stages: header, payload, checksum, and completion. The FSM transitions through multiple states as indicated by external control signals, and generates status flags (**valid_pkt**, **error_pkt**) to indicate whether a packet was successfully received or failed due to checksum errors.

Signal	Direction	Width	Description
clk	Input	1	System clock. All state transitions occur on the rising edge.
reset	Input	1	Synchronous reset. Forces the FSM back to IDLE and clears outputs.
start_pkt	Input	1	Initiates packet reception. Valid only in IDLE.
hdr_done	Input	1	Signals completion of header processing. Valid only in HEADER.
payload_done	Input	1	Signals completion of payload processing. Valid only in PAYLOAD.
chk_ok	Input	1	Indicates checksum verification passed. Valid only in CHECKSUM.
chk_fail	Input	1	Indicates checksum verification failed. Valid only in CHECKSUM.
abort	Input	1	Aborts packet reception at any stage, returning FSM to IDLE.
valid_pkt	Output	1	Pulses high when packet successfully received (CHECKSUM to DONE).
error_pkt	Output	1	Pulses high when checksum fails (CHECKSUM to IDLE).
state	Output	3	Encoded FSM state (0-4).

Table 1: Packet Module I/O Interface Signals

1.1 State Encoding

The FSM implements five distinct states, each representing a stage in packet reception:

- **IDLE**: Waiting for packet start.
- **HEADER**: Processing header.
- **PAYLOAD**: Processing payload.
- **CHECKSUM**: Verifying checksum.

- **DONE:** Packet reception complete; automatically returns to IDLE.

1.2 Behavioral description

The packet reception FSM operates according to the following behavioral rules:

1. On reset assertion, the FSM enters the IDLE state and all output signals are cleared.
2. In the IDLE state, assertion of the `start_pkt` signal causes a transition to the HEADER state.
3. In the HEADER state, assertion of the `hdr_done` signal causes a transition to the PAYLOAD state.
4. In the PAYLOAD state, assertion of the `payload_done` signal causes a transition to the CHECKSUM state.
5. In the CHECKSUM state, two conditions are possible:
 - If `chk_ok` is asserted, the FSM transitions to the DONE state and the `valid_pkt` output pulses high.
 - If `chk_fail` is asserted, the FSM returns to the IDLE state and the `error_pkt` output pulses high.
6. In the DONE state, the FSM automatically transitions back to IDLE on the next clock cycle.
7. At any stage in the FSM, assertion of the `abort` signal forces the FSM back to IDLE with all output signals cleared.

1.3 Finite State Machine Graph

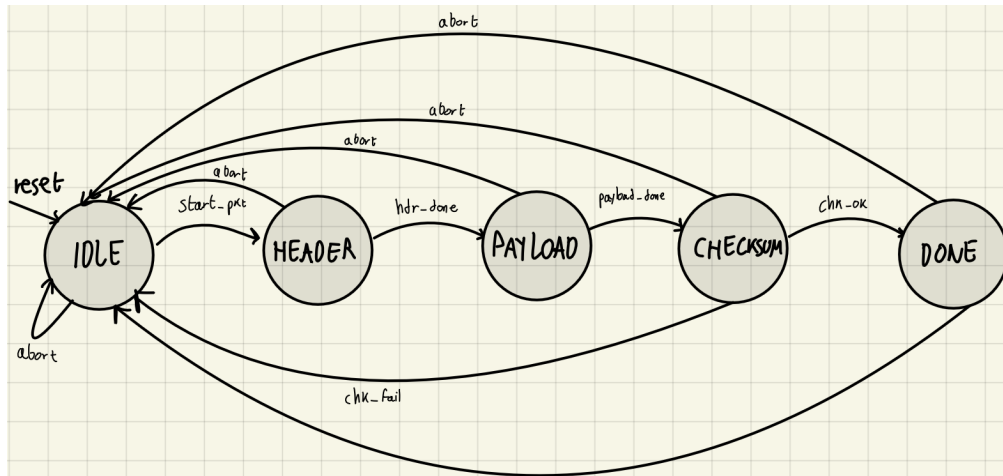


Figure 1: Finite State Machine Graph for Packet Reception

1.4 Formal specification

1.4.1 List and Description of Assertions and Assumptions

This section provides a detailed list and description of the formal properties added to the `packet.sv` module, categorized by their functional purpose.

1.4.2 Environment Assumptions (`assume`)

These properties constrain the verification environment to ensure the formal tool generates valid and realistic input sequences.

- **initial assume(reset);**
Forces the formal engine to start the verification process with the system in an active reset state, establishing a valid initial baseline.


```
initial assume(reset);
```
- **flags_zero_after_reset**
Ensures that the input verification flags `chk_ok` and `chk_fail` remain deactivated (0) in the cycle immediately following a reset condition.


```
flags_zero_after_reset: assume property (  
    @(posedge clk) reset | => (chk_ok == 1'b0 && chk_fail == 1'b0)  
);
```
- **only_one_chk_flag**
Constrains the environment so that when the FSM is evaluating the checksum (`state == CHECKSUM`), it can assert either `chk_ok` or `chk_fail`, but never both simultaneously or neither (`chk_ok ≠ chk_fail`).


```
only_one_chk_flag: assume property (  
    @(posedge clk) disable iff (reset) (state == CHECKSUM) | -> (chk_ok != chk_fail)  
);
```

1.4.3 Reset & Structural Assertions (`assert`)

These properties verify that the foundational structural rules and initialization of the FSM are working correctly.

- **idle_after_reset**
Verifies that a high `reset` signal successfully forces the FSM into the `IDLE` state on the following clock edge.


```
idle_after_reset: assert property (  
    @(posedge clk) reset | => (state == IDLE)  
);
```
- **always_a_state**
A safety property proving that the FSM can never drop into an undefined or illegal state; it must strictly dwell within one of the five defined operational states (`IDLE`, `HEADER`, `PAYLOAD`, `CHECKSUM`, or `DONE`).


```
always_a_state: assert property (  
    @(posedge clk) disable iff (reset)  
        state == IDLE      ||  
        state == HEADER    ||  
        state == PAYLOAD   ||  
        state == CHECKSUM  ||  
        state == DONE  
);
```

1.4.4 Output Consistency Assertions (assert)

These properties check that the output status flags are strictly synchronized and aligned with the correct internal FSM states.

- **done_if_valid**

Guarantees that if the output flag `valid_pkt` is asserted high, the FSM is concurrently in the DONE state.

```
done_if_valid: assert property (
    @(posedge clk) disable iff (reset)
    valid_pkt |-> (state == DONE)
);
```

- **idle_if_error**

Guarantees that if the output flag `error_pkt` is asserted high, the FSM has correctly returned to the IDLE state.

```
idle_if_error: assert property (
    @(posedge clk) disable iff (reset)
    error_pkt |-> (state == IDLE)
);
```

1.4.5 State Transition Assertions (assert)

These assertions verify the forward progress paths of the FSM pipeline under proper input stimuli.

- **header_after_idle**

Verifies that asserting `start_pkt` while in IDLE (without an abort) advances the FSM to the HEADER state.

```
header_after_idle:    assert property (
    @(posedge clk) disable iff (reset)
    start_pkt && (state == IDLE) && !abort |=> (state == HEADER)
);
```

- **payload_after_header**

Verifies that asserting `hdr_done` while in HEADER (without an abort) advances the FSM to the PAYLOAD state.

```
payload_after_header:    assert property (
    @(posedge clk) disable iff (reset)
    hdr_done && (state == HEADER) && !abort |=> (state == PAYLOAD)
);
```

- **checksum_after_payload**

Verifies that asserting `payload_done` while in PAYLOAD (without an abort) advances the FSM to the CHECKSUM state.

```
checksum_after_payload:    assert property (
    @(posedge clk) disable iff (reset)
    payload_done && (state == PAYLOAD) && !abort |=> (state == CHECKSUM)
);
```

- **done_after_checksum_ok**

Verifies that if the checksum passes (`chk_ok`) while in CHECKSUM (without an abort), the FSM advances to the DONE state.

```
done_after_checksum_ok:    assert property (
    @(posedge clk) disable iff (reset)
    chk_ok && (state == CHECKSUM) && !abort |=> (state == DONE)
);
```

- **idle_after_checksum_fail**

Verifies that if the checksum fails (`chk_fail`) while in `CHECKSUM` (without an `abort`), the FSM drops the packet and returns to `IDLE`.

```
idle_after_checksum_fail: assert property (
    @(posedge clk) disable iff (reset)
    chk_fail && (state == CHECKSUM) && !abort |>= (state == IDLE)
);
```

- **idle_after_done**

Verifies the automatic, single-cycle transition from the `DONE` state back to `IDLE` without requiring any external stimulus.

```
idle_after_done: assert property (
    @(posedge clk) disable iff (reset)
    (state == DONE) && !abort |>= (state == IDLE)
);
```

- **idle_after_abort**

Acts as a global safety check verifying that the assertion of the `abort` signal immediately overrides any state and forces the FSM back to `IDLE` on the next cycle.

```
idle_after_abort: assert property (
    @(posedge clk) disable iff (reset)
    abort |>= (state == IDLE)
);
```

1.4.6 State Holding Assertions (`assert`)

These safety properties prove that the FSM will not advance prematurely and will stay in its current state until the exact progression or override conditions are met.

- **stay_in_idle**

Ensures the FSM holds in the `IDLE` state as long as no packet start (`start_pkt`) or system `abort` is requested.

```
stay_in_idle: assert property(
    @(posedge clk) disable iff (reset)
    (state == IDLE) && !start_pkt && !abort |>= (state == IDLE)
);
```

- **stay_in_header_if_not_done**

Ensures the FSM holds in the `HEADER` state until processing is explicitly finished via `hdr_done` or interrupted by an `abort`.

```
stay_in_header_if_not_done: assert property (
    @(posedge clk) disable iff (reset)
    (state == HEADER) && !hdr_done && !abort |>= (state == HEADER)
);
```

- **stay_in_payload_if_not_done**

Ensures the FSM holds in the `PAYLOAD` state until processing is explicitly finished via `payload_done` or interrupted by an `abort`.

```
stay_in_payload_if_not_done: assert property (
    @(posedge clk) disable iff (reset)
    (state == PAYLOAD) && !payload_done && !abort |>= (state == PAYLOAD)
);
```

- **stay_in_checksum**

Ensures the FSM holds in the `CHECKSUM` state as long as `chk_ok`, `chk_fail`, and `abort` all remain deasserted.

```

stay_in_checksum: assert property(
    @(posedge clk) disable iff (reset)
    (state == CHECKSUM) && !chk_fail && !chk_ok && !abort |=> (state == CHECKSUM)
);

```

1.4.7 Code

The full code snippet contains all the formal properties implemented in the `packet.sv` can be viewed in **Appendix A**.

1.5 EBMC output

The output of the EBMC tool after running the verification on the `packet.sv` module is presented in **Appendix B**. The results indicate that all properties have been successfully proved up to the bound of 20 cycles, and no counterexamples were found, This confirms the correctness of the finite state machine design with respect to the specified properties.

2 Bug Injection and Re-verification

I have then introduced a bug in the packet design, by setting, when reset is not active, `valid_pkt` and `error_pkt` to 1, regardless of the state of the FSM:

```

if (reset) begin
    state    <= IDLE;
    valid_pkt <= 0;
    error_pkt <= 0;
end else begin
    valid_pkt <= 1;
    error_pkt <= 1;

// rest of the code

```

After running the EBMC tool again, the properties `done_if_valid` and `idle_if_error` have been refuted, and the two counterexamples provided by EBMC show that both properties fail when the FSM is in the CHECKSUM state, and the `abort` signal is asserted, which forces the FSM to return to IDLE while both `valid_pkt` and `error_pkt` are high. After that, the FSM transitions to the HEADER state due to the assertion of `start_pkt`, and both flags remain high, which causes the failure of both properties, as the FSM is not in the expected states for either flag to be high.

In fact, when `valid_pkt` is high, the FSM should be in the DONE state, and when `error_pkt` is high, the FSM should be in the IDLE state, but due to the bug, both flags are high while the FSM is in the CHECKSUM state, which is an invalid state for both flags to be asserted.

Finally, after a reset, this behavior is not observed, as the reset code has not been changed.

The full output of the EBMC tool after introducing the bug can be viewed in **Appendix C**.

In the next page, the waveforms of the counterexamples are shown.

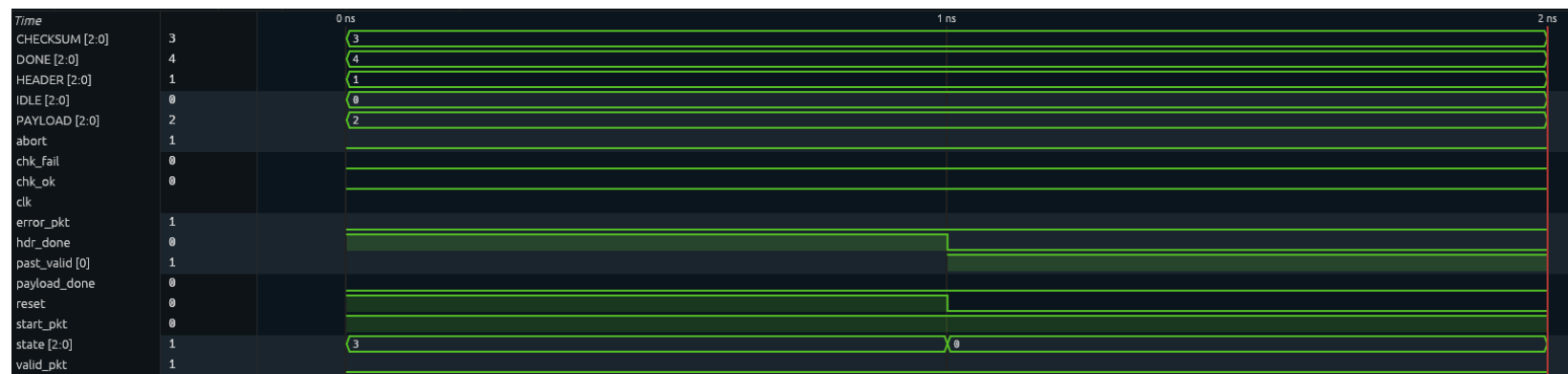


Figure 2: Simulation waveforms

3 Conclusion

In this laboratory, I have formally specified the behavior of a packet reception FSM using SystemVerilog assertions and assumptions, and I have used the EBMC tool to verify that the properties hold for the design. After confirming that all properties were proved, I introduced a bug in the design and re-ran the verification, which successfully refuted two properties and provided counterexamples that demonstrate the failure of these properties due to the introduced bug.

4 Appendix

To show the waveforms, I have used the Surfer software.

4.1 Number of hours required

For the completion of this laboratory, it has been needed 10 hours.

4.2 Use of AI

Gemini AI has been used to fix bugs in the code and to improve some sentences in the report.

4.3 Appendix A

```
initial assume(reset);

idle_after_reset: assert property (
    @(posedge clk) reset | => (state == IDLE)
);

flags_zero_after_reset: assume property (
    @(posedge clk) reset | => (chk_ok == 1'b0 && chk_fail == 1'b0)
);

only_one_chk_flag: assume property (
    @(posedge clk) disable iff (reset) (state == CHECKSUM) | -> (chk_ok !=
        ↪ chk_fail)
);

//ALWAYS A STATE

always_a_state: assert property (
    @(posedge clk) disable iff (reset)
        state == IDLE ||
        state == HEADER ||
        state == PAYLOAD ||
        state == CHECKSUM ||
        state == DONE
);

// CORRECT STATE

done_if_valid: assert property (
    @(posedge clk) disable iff (reset)
        valid_pkt | -> (state == DONE)
);

idle_if_error: assert property (
    @(posedge clk) disable iff (reset)
        error_pkt | -> (state == IDLE)
);

// STATE TRANSITIONS

header_after_idle:          assert property (
    @(posedge clk) disable iff (reset)
        start_pkt && (state == IDLE) && !abort          | => (state == HEADER)
```

```

);

payload_after_header:    assert property (
    @(posedge clk) disable iff (reset)
    hdr_done && (state == HEADER) && !abort    |> (state == PAYLOAD)
);

checksum_after_payload:  assert property (
    @(posedge clk) disable iff (reset)
    payload_done && (state == PAYLOAD) && !abort    |> (state == CHECKSUM)
);

done_after_checksum_ok:  assert property (
    @(posedge clk) disable iff (reset)
    chk_ok && (state == CHECKSUM) && !abort    |> (state == DONE)
);

idle_after_checksum_fail: assert property (
    @(posedge clk) disable iff (reset)
    chk_fail && (state == CHECKSUM) && !abort    |> (state == IDLE)
);

idle_after_done:         assert property (
    @(posedge clk) disable iff (reset)
    (state == DONE) && !abort    |> (state == IDLE)
);

idle_after_abort:        assert property (
    @(posedge clk) disable iff (reset)
    abort    |> (state == IDLE)
);

// STAY IN A STATE UNTIL THE PROPER SIGNALS
stay_in_idle: assert property(
    @(posedge clk) disable iff (reset)
    (state == IDLE) && !start_pkt && !abort |> (state == IDLE)
);

stay_in_header_if_not_done: assert property (
    @(posedge clk) disable iff (reset)
    (state == HEADER) && !hdr_done && !abort |> (state == HEADER)
);

stay_in_payload_if_not_done: assert property (
    @(posedge clk) disable iff (reset)
    (state == PAYLOAD) && !payload_done && !abort |> (state == PAYLOAD)
);

stay_in_checksum: assert property(
    @(posedge clk) disable iff (reset)
    (state == CHECKSUM) && !chk_fail && !chk_ok && !abort |> (state == CHECKSUM)
);

```

4.4 Appendix B

```
./run.sh
Parsing packet.sv
Converting
Type-checking Verilog::packet
Generating Decision Problem
Using MiniSAT 2.2.1 with simplifier
Properties
Solving with propositional reduction
SAT checker: instance is UNSATISFIABLE
UNSAT: No path found within bound

** Results:
[packet.assume.1] assume packet.reset: ASSUMED
[packet.idle_after_reset] always (packet.reset ==> packet.state == packet.IDLE): PROVED
up to bound 20
[packet.flags_zero_after_reset] always (packet.reset ==> packet.chk_ok == 1'b0 &&
packet.chk_fail == 1'b0): ASSUMED
[packet.only_one_chk_flag] always (disable iff (packet.reset) packet.state == packet.
CHECKSUM ==> packet.chk_ok != packet.chk_fail): ASSUMED
[packet.always_a_state] always (disable iff (packet.reset) packet.state == packet.IDLE
|| packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet.state
== packet.CHECKSUM || packet.state == packet.DONE): PROVED up to bound 20
[packet.done_if_valid] always (disable iff (packet.reset) packet.valid_pkt ==> packet.
state == packet.DONE): PROVED up to bound 20
[packet.idle_if_error] always (disable iff (packet.reset) packet.error_pkt ==> packet.
state == packet.IDLE): PROVED up to bound 20
[packet.header_after_idle] always (disable iff (packet.reset) packet.start_pkt &&
packet.state == packet.IDLE && !packet.abort ==> packet.state == packet.HEADER):
PROVED up to bound 20
[packet.payload_after_header] always (disable iff (packet.reset) packet.hdr_done &&
packet.state == packet.HEADER && !packet.abort ==> packet.state == packet.PAYLOAD):
PROVED up to bound 20
[packet.checksum_after_payload] always (disable iff (packet.reset) packet.payload_done
&& packet.state == packet.PAYLOAD && !packet.abort ==> packet.state == packet.
CHECKSUM): PROVED up to bound 20
[packet.done_after_checksum_ok] always (disable iff (packet.reset) packet.chk_ok &&
packet.state == packet.CHECKSUM && !packet.abort ==> packet.state == packet.DONE):
PROVED up to bound 20
[packet.idle_after_checksum_fail] always (disable iff (packet.reset) packet.chk_fail &&
packet.state == packet.CHECKSUM && !packet.abort ==> packet.state == packet.IDLE):
PROVED up to bound 20
[packet.idle_after_done] always (disable iff (packet.reset) packet.state == packet.DONE
&& !packet.abort ==> packet.state == packet.IDLE): PROVED up to bound 20
[packet.idle_after_abort] always (disable iff (packet.reset) packet.abort ==> packet.
state == packet.IDLE): PROVED up to bound 20
[packet.stay_in_idle] always (disable iff (packet.reset) packet.state == packet.IDLE &&
!packet.start_pkt && !packet.abort ==> packet.state == packet.IDLE): PROVED up to
bound 20
[packet.stay_in_header_if_not_done] always (disable iff (packet.reset) packet.state ==
packet.HEADER && !packet.hdr_done && !packet.abort ==> packet.state == packet.
HEADER): PROVED up to bound 20
[packet.stay_in_payload_if_not_done] always (disable iff (packet.reset) packet.state ==
packet.PAYLOAD && !packet.payload_done && !packet.abort ==> packet.state == packet.
PAYLOAD): PROVED up to bound 20
[packet.stay_in_checksum] always (disable iff (packet.reset) packet.state == packet.
CHECKSUM && !packet.chk_fail && !packet.chk_ok && !packet.abort ==> packet.state ==
packet.CHECKSUM): PROVED up to bound 20
```

5 Appendix C

```
./run.sh
Parsing packet.sv
Converting
Type-checking Verilog::packet
Generating Decision Problem
Using MiniSAT 2.2.1 with simplifier
Properties
Solving with propositional reduction
SAT checker: instance is SATISFIABLE
SAT: path found
SAT checker: instance is UNSATISFIABLE
UNSAT: No path found within bound

** Results:
[packet.assume.1] assume packet.reset: ASSUMED
[packet.idle_after_reset] always (packet.reset ==> packet.state == packet.IDLE): PROVED
up to bound 20
[packet.flags_zero_after_reset] always (packet.reset ==> packet.chk_ok == 1'b0 &&
packet.chk_fail == 1'b0): ASSUMED
[packet.only_one_chk_flag] always (disable iff (packet.reset) packet.state == packet.
CHECKSUM ==> packet.chk_ok != packet.chk_fail): ASSUMED
[packet.always_a_state] always (disable iff (packet.reset) packet.state == packet.IDLE
|| packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet.state
== packet.CHECKSUM || packet.state == packet.DONE): PROVED up to bound 20
[packet.done_if_valid] always (disable iff (packet.reset) packet.valid_pkt ==> packet.
state == packet.DONE): REFUTED
Counterexample:

Transition system state 0
-----
packet.clk = ?
packet.reset = 1
packet.start_pkt = 1
packet.hdr_done = 1
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
packet.abort = 0
packet.valid_pkt = 0
packet.error_pkt = 0
packet.state = 3'b011 (011)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = ?

Transition system state 1
-----
packet.clk = ?
packet.reset = 0
packet.start_pkt = 1
packet.hdr_done = 0
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
```

```

packet.abort = 0
packet.valid_pkt = 0
packet.error_pkt = 0
packet.state = 3'b000 (000)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = 1'b1

```

Transition system state 2

```

packet.clk = ?
packet.reset = 0
packet.start_pkt = 0
packet.hdr_done = 0
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
packet.abort = 1
packet.valid_pkt = 1
packet.error_pkt = 1
packet.state = 3'b001 (001)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = 1'b1

```

[packet.idle_if_error] always (disable iff (packet.reset) packet.error_pkt |-> packet.
state == packet.IDLE): **REFUTED**
Counterexample:

Transition system state 0

```

packet.clk = ?
packet.reset = 1
packet.start_pkt = 1
packet.hdr_done = 1
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
packet.abort = 0
packet.valid_pkt = 0
packet.error_pkt = 0
packet.state = 3'b011 (011)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = ?

```

Transition system state 1

```

packet.clk = ?
packet.reset = 0

```

```

packet.start_pkt = 1
packet.hdr_done = 0
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
packet.abort = 0
packet.valid_pkt = 0
packet.error_pkt = 0
packet.state = 3'b000 (000)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = 1'b1

```

Transition system state 2

```

-----
packet.clk = ?
packet.reset = 0
packet.start_pkt = 0
packet.hdr_done = 0
packet.payload_done = 0
packet.chk_ok = 0
packet.chk_fail = 0
packet.abort = 1
packet.valid_pkt = 1
packet.error_pkt = 1
packet.state = 3'b001 (001)
packet.IDLE = 3'b000 (000)
packet.HEADER = 3'b001 (001)
packet.PAYLOAD = 3'b010 (010)
packet.CHECKSUM = 3'b011 (011)
packet.DONE = 3'b100 (100)
packet.past_valid = 1'b1

```

```

[packet.header_after_idle] always (disable iff (packet.reset) packet.start_pkt &&
    packet.state == packet.IDLE && !packet.abort |>= packet.state == packet.HEADER):
    PROVED up to bound 20
[packet.payload_after_header] always (disable iff (packet.reset) packet.hdr_done &&
    packet.state == packet.HEADER && !packet.abort |>= packet.state == packet.PAYLOAD):
    PROVED up to bound 20
[packet.checksum_after_payload] always (disable iff (packet.reset) packet.payload_done
    && packet.state == packet.PAYLOAD && !packet.abort |>= packet.state == packet.
    CHECKSUM): PROVED up to bound 20
[packet.done_after_checksum_ok] always (disable iff (packet.reset) packet.chk_ok &&
    packet.state == packet.CHECKSUM && !packet.abort |>= packet.state == packet.DONE):
    PROVED up to bound 20
[packet.idle_after_checksum_fail] always (disable iff (packet.reset) packet.chk_fail &&
    packet.state == packet.CHECKSUM && !packet.abort |>= packet.state == packet.IDLE):
    PROVED up to bound 20
[packet.idle_after_done] always (disable iff (packet.reset) packet.state == packet.DONE
    && !packet.abort |>= packet.state == packet.IDLE): PROVED up to bound 20
[packet.idle_after_abort] always (disable iff (packet.reset) packet.abort |>= packet.
    state == packet.IDLE): PROVED up to bound 20
[packet.stay_in_idle] always (disable iff (packet.reset) packet.state == packet.IDLE &&
    !packet.start_pkt && !packet.abort |>= packet.state == packet.IDLE): PROVED up to
    bound 20

```

```
[packet.stay_in_header_if_not_done] always (disable iff (packet.reset) packet.state ==
packet.HEADER && !packet.hdr_done && !packet.abort |> packet.state == packet.
HEADER): PROVED up to bound 20
[packet.stay_in_payload_if_not_done] always (disable iff (packet.reset) packet.state ==
packet.PAYLOAD && !packet.payload_done && !packet.abort |> packet.state == packet
.PAYLOAD): PROVED up to bound 20
[packet.stay_in_checksum] always (disable iff (packet.reset) packet.state == packet.
CHECKSUM && !packet.chk_fail && !packet.chk_ok && !packet.abort |> packet.state ==
packet.CHECKSUM): PROVED up to bound 20
```